

API Testing and Automation Course Syllabus

Archer Infotech, Kothrud Pune | Classroom & live-online batches | Updated September 2026

Institute: Archer Infotech, Kothrud, Pune - IT training since 2009

Website: <https://archerinfotech.in>

Contact: +91 9850 678451 | info@archerinfotech.in

Mode: Classroom at Kothrud, Pune and live-online batches

Duration: 2 Months

Level: Intermediate

Course page: <https://archerinfotech.in/courses/testing-qa/api-testing-training-in-pune>

Course Overview

API testing is the fastest way for a tester to become genuinely valuable, because it catches defects before they ever reach a screen. This 2-month programme starts with HTTP and REST from first principles, builds Postman to professional depth, covers authentication from API keys through OAuth 2.0, then has you build an automation framework in your choice of Rest Assured, pytest or Playwright — finishing with contract testing, API security fundamentals, CI pipelines and a full AI-assisted testing module.

Why Learn API Testing in 2026

Modern applications are assemblies of services, and most defects live in the joins between them rather than in the interface on top. An API test runs in milliseconds where a UI test takes seconds, does not break when a button moves, and can exercise error paths, boundary values and permission rules that are painful or impossible to trigger through a screen. That is why the test pyramid puts far more tests at the service layer than at the UI layer, and why teams that invest in API testing ship faster with fewer regressions.

For a tester's career the calculation is straightforward. API testing is the single highest-leverage skill a manual tester can add: the first half of this course needs no programming at all, yet it immediately makes you useful on integration work, and the second half moves you into automation on whichever language your target team uses. Almost every Pune SDET job description now lists API testing alongside UI automation, and candidates who can only click through a browser are competing in the most crowded part of the market.

What has changed recently is scope. API testing in 2026 means more than sending a request and checking a status code: schema validation against an OpenAPI contract, contract testing between services, rate-limit and pagination behaviour, and the OWASP API security basics that broken authorisation defects keep making relevant. AI has also landed here faster than in UI testing, because an OpenAPI specification is a machine-readable description a model can generate cases from — which is exactly what the final module covers, along with how to check what it produces.

- The first half needs no programming — accessible to any manual tester
- HTTP and REST taught from first principles, not assumed
- Postman to professional depth, including scripting and the Collection Runner
- Choose your automation stack — Rest Assured, pytest or Playwright
- Schema validation, contract testing and OWASP API security fundamentals
- A full AI-assisted module built around generating tests from OpenAPI specs

Who This Course Is For

This course fits you if

- Manual QA testers looking for the highest-leverage skill to add first

- UI automation testers broadening into the service layer to move towards SDET roles
- BE, BCA, BSc-CS or MCA graduates targeting QA and SDET roles in Pune
- Backend developers taking on test ownership for their own services
- Support and integration engineers who debug API issues and want to test them properly

This course is not the right fit if

- Anyone wanting a pure manual-testing course — take Software Testing & QA instead
- Engineers seeking a deep performance-engineering course; load testing here is fundamentals only
- Anyone wanting full API penetration testing — security here is a tester's grounding, not an offensive-security course
- Testers who want only UI automation, where the Selenium or Playwright tracks fit better

Detailed Curriculum

Module 1: Software Testing and API Fundamentals (Week 1)

The mental model everything else depends on. Client-server architecture, what an API actually is, web services, and REST as an architectural style rather than a synonym for JSON over HTTP — including an honest comparison with SOAP, which still runs in plenty of Indian banking and insurance systems.

Then HTTP itself, in the detail a tester needs: the request and response anatomy, methods and their idempotency, status codes and what each class actually promises, headers, query and path parameters, and cookies. JSON and XML close the module, since those are the payloads you will spend two months asserting against.

- Introduction to software testing
- Client-server architecture
- APIs and web services
- REST architecture and its constraints
- REST versus SOAP
- HTTP and HTTPS
- HTTP request and response anatomy
- HTTP methods and idempotency
- Status codes and what each class promises
- Headers and content negotiation
- Query parameters and path parameters
- Cookies and sessions
- JSON structure and parsing
- XML structure and parsing

Module 2: Manual API Testing with Postman (Weeks 1–2)

Postman used the way a professional tester uses it, not the way a tutorial demonstrates it. Installation and setup, then building requests across every method — GET, POST, PUT, PATCH, DELETE — with headers, query parameters and request bodies constructed deliberately.

Response validation covers status, headers, body and response time. Then the organisational layer that separates a working tester from someone clicking Send: collections that model a real API surface, environments for dev, staging and production, and the variable hierarchy — global, collection, environment and local — which is the thing most people get subtly wrong and then debug for an hour.

- Postman installation and setup
- Creating requests and saving them
- GET requests and query parameters
- POST requests and request bodies
- PUT and PATCH requests
- DELETE requests
- Headers and content types
- Response validation — status, headers, body, timing

- Collections that model a real API surface
- Environments for dev, staging and production
- Variables — global, collection, environment and local
- Variable precedence and common mistakes

Module 3: API Authentication (Week 3)

The subject that blocks more new API testers than any other, because every real API is protected and every protection scheme works differently. Basic authentication and why it survives only behind TLS, API keys and where they belong in a request, and Bearer tokens.

JWT gets proper treatment — structure, claims, expiry and what a tester can and cannot verify from a token — followed by OAuth 2.0 and its grant types, including the authorisation-code flow you will meet most often. The module ends on the distinction that produces the most severe real defects: authentication testing (are you who you claim) versus authorisation testing (are you allowed to do this).

- Basic authentication and its constraints
- API keys and where they belong in a request
- Bearer tokens
- JWT structure, claims and expiry
- OAuth 2.0 and its grant types
- The authorisation-code flow in practice
- Token refresh and expiry handling
- Authentication testing
- Authorisation testing and privilege escalation cases

Module 4: Postman Automation (Week 3)

Turning a collection into an executable regression suite without leaving Postman — the step that makes API testing repeatable for testers who are not yet writing framework code.

Postman scripting with JavaScript: pre-request scripts for setup and token acquisition, and test scripts with `pm.test` assertions against status, schema and body. Variables carry values between requests, dynamic data handles timestamps and unique identifiers, and request chaining lets one call feed the next — create a resource, capture its id, use it, delete it. The Collection Runner then executes the whole suite, including data-driven runs from CSV or JSON.

- Postman scripting with JavaScript
- Pre-request scripts for setup and tokens
- Test scripts and `pm.test` assertions
- Asserting on status, body and response time
- Setting and reading variables between requests
- Dynamic data and unique identifiers
- Chaining API requests into a workflow
- The Collection Runner
- Data-driven API testing from CSV and JSON

Module 5: API Documentation (Week 4)

Reading the specification is the fastest route to good test coverage, and most testers never learn to do it properly. Swagger and the OpenAPI specification: how an API describes its own paths, parameters, request bodies, responses and schemas.

The module teaches you to derive test cases directly from a spec — every enum is a set of valid values and an invalid one, every required field is a negative case, every documented status code is an expected outcome. Testing APIs straight from Swagger UI, API contracts, and request and response schemas close it, setting up the schema validation and contract testing that arrive in Module 7.

- Swagger and Swagger UI
- The OpenAPI specification structure
- Reading API documentation critically

- Deriving test cases directly from a spec
- Testing APIs from Swagger UI
- API contracts and their guarantees
- Request and response schemas
- Spotting undocumented and inconsistent behaviour

Module 6: API Automation Framework (Weeks 5–6)

The module where you stop clicking and start engineering. You pick one of three stacks and build a real framework in it: Rest Assured with Java if you are heading for Java teams, Python Requests with PyTest if you prefer Python or already know it, or Playwright API testing with TypeScript if you also want UI automation in the same tool. The concepts are identical across all three, so the choice is about your target job, not difficulty.

Request and response specifications remove repetition, serialisation and deserialisation map payloads to data models, and assertions verify structure as well as values. Then the framework layer: parameterisation and data-driven testing, reusable API clients, configuration management, authentication handling in one place, logging, reporting, and an architecture a new joiner can navigate.

- Choosing a stack — Rest Assured, PyTest or Playwright
- Creating API automation tests
- Request specification and shared setup
- Response specification and reusable validation
- Assertions on structure and values
- Serialization and deserialization
- POJOs and typed data models
- Parameterisation
- Data-driven testing
- Reusable API clients
- Configuration management across environments
- Centralised authentication handling
- Logging and reporting
- Framework architecture and folder design

Module 7: Advanced API Testing (Week 7)

The coverage that separates a competent API tester from a thorough one. Schema validation asserts the shape of a response, not just its values — the check that catches a backend change before it breaks three consumers. Contract testing extends that between services.

Then the cases that matter and get skipped: negative testing, boundary testing, error handling and error-response consistency, rate-limit behaviour, pagination correctness across page boundaries, and file-upload APIs. Database validation confirms that a successful response actually persisted what it claimed, and end-to-end API workflows chain multiple services into the business flows a user would recognise.

- Schema validation against JSON Schema
- Contract testing between services
- Negative testing and invalid input
- Boundary testing
- Error handling and error-response consistency
- Rate limit testing
- Pagination testing across page boundaries
- File upload APIs
- API chaining and stateful workflows
- Dynamic data handling
- Database validation of persisted state
- End-to-end API workflows

Module 8: API Performance and Security Fundamentals (Week 7)

Two adjacent disciplines at the depth a QA engineer is actually expected to own. On performance: validating response times against a stated budget, basic load testing to see how an endpoint behaves under concurrency, and rate limiting as both a functional behaviour and a protection mechanism.

On security: authentication vulnerabilities, authorisation issues including the broken-object-level-authorisation defect that dominates real API breaches, input validation, injection testing, and sensitive data exposure in responses and logs — framed by the OWASP API Security list. This is a tester's working grounding, deliberately not a penetration-testing course.

- API response-time validation against a budget
- Basic load testing and concurrency behaviour
- Rate limiting as behaviour and as protection
- Authentication vulnerabilities
- Authorisation issues and broken object-level authorisation
- OWASP API Security fundamentals
- Input validation testing
- Injection testing basics
- Sensitive data exposure in responses and logs

Module 9: Git and CI/CD (Week 8)

Making the suite run without you. Git and GitHub for automation projects — branching, pull requests and review applied to test code — then Newman, which runs a Postman collection from the command line and is the bridge that lets non-framework work still run in CI.

Jenkins and GitHub Actions cover the pipeline itself: triggering on push and pull request, environment-based execution so the same suite runs against dev, staging and production with different variables, secret handling for tokens, scheduled regression runs, and publishing test reports where a delivery manager can read them.

- Git and GitHub for automation projects
- Managing API automation projects and secrets hygiene
- Newman for command-line collection runs
- Jenkins jobs and pipelines
- GitHub Actions workflows
- Automated API testing in CI/CD
- Environment-based execution
- Secret and token handling in pipelines
- Publishing test reports

Module 10: AI-Assisted API Testing (Week 8 + capstone)

API testing is where generative AI has landed hardest, because an OpenAPI specification is a machine-readable contract a model can reason over directly. Generating test scenarios straight from a spec, drafting Postman tests and automation scripts, and producing positive, negative and boundary cases — the last being the coverage humans most reliably skip.

Then payload and synthetic data generation, AI-assisted response and error analysis, understanding unfamiliar API documentation, generating mocks, and contract-testing support. The module keeps a validation discipline throughout: a generated test that passes may be asserting nothing, and a generated negative case may encode the wrong expectation. It closes on prompt engineering for API testers, coding agents for automation, and how to review AI-generated tests before they enter a suite others rely on.

- Generative AI for API testing — capabilities and limits
- Generating API test scenarios from OpenAPI specifications
- AI-assisted Postman test generation
- Generating API automation scripts
- Creating positive and negative test cases with AI
- AI-generated boundary test cases
- AI-assisted request payload generation
- Synthetic test data generation

- API response analysis using AI
- Automated error analysis
- AI-assisted API documentation understanding
- Generating API mocks with AI
- AI-assisted contract testing
- AI-based defect analysis
- Generating Rest Assured, PyTest and Playwright API tests
- Prompt engineering for API test engineers
- Using coding agents for API automation
- Validating AI-generated API tests before they ship

Capstone Projects

Postman Regression Suite for a Public REST API

A complete Postman collection against a public REST API — full CRUD coverage, environments for two stages, a variable strategy that survives reordering, pre-request token acquisition, and assertions on status, schema and response time. Runs end to end in the Collection Runner and from the command line with Newman. The project a manual tester can complete before writing any framework code.

- Technology: Postman
- Technology: Newman
- Technology: JavaScript assertions
- Technology: JSON Schema
- Technology: Environments and variables

API Automation Framework in Your Chosen Stack

A layered framework built in Rest Assured, PyTest or Playwright — reusable API clients, request and response specifications, typed data models, centralised authentication, environment configuration, structured logging and reporting. Includes schema validation on every response and a negative-test suite covering invalid input, missing authorisation and boundary values.

- Technology: Rest Assured / PyTest / Playwright
- Technology: JSON Schema validation
- Technology: Data models
- Technology: Config management
- Technology: Allure or equivalent reporting

Capstone — Contract-Tested API Suite in CI with AI-Assisted Coverage

The capstone runs the framework in Jenkins or GitHub Actions on every push, against environment-specific configuration with secrets injected rather than committed. It adds contract testing against an OpenAPI specification, database validation of persisted state, rate-limit and pagination coverage, and a documented AI-assisted workflow — cases generated from the spec, then reviewed, corrected and validated by you, with review notes committed alongside.

- Technology: Rest Assured / PyTest / Playwright
- Technology: OpenAPI contract testing
- Technology: Database validation
- Technology: Jenkins or GitHub Actions
- Technology: Newman
- Technology: AI coding assistants

Career Outcomes

API testing is the skill that most reliably changes a QA candidate's position in the Pune market, because it moves you from the crowded manual-testing pool into the integration and automation pool where demand is higher and supply thinner.

Services companies — Persistent, Capgemini, LTIMindtree, Tech Mahindra, Cognizant, Wipro, Accenture, Amdocs — run

large integration-testing engagements for banking, insurance and telecom clients where API coverage is the core of the work. Product companies expect it as a baseline part of any SDET role.

The most common outcome we see is not a job titled 'API Test Engineer' but a stronger offer for a general automation or SDET role, because the candidate can cover both layers. That breadth is also what shortens the path to the senior bands: a tester who can validate a contract, read an OpenAPI spec, check authorisation logic and wire it all into CI is doing work that overlaps with backend engineering. We do not claim guaranteed placement — our institute-records placement rate is 90% across all tracks.

Roles after this course

- API Test Engineer
- QA Automation Engineer
- SDET (Software Development Engineer in Test)
- Integration Test Engineer
- Backend Test Engineer

Indicative salary bands

- API / QA Automation Engineer (fresher) - Rs. 3.5–5.5 LPA (source: AmbitionBox Pune QA Automation (last 12 mo))
- API Test Engineer (1–3 yrs) - Rs. 6–10 LPA (source: Indeed Pune API testing listings (last 12 mo))
- SDET (3–6 yrs) - Rs. 10–18 LPA (source: Glassdoor Pune SDET)
- Test Architect / Sr SDET (6+ yrs) - Rs. 14–22 LPA (source: AmbitionBox Pune Test Architect)

Companies hiring for these skills

- Persistent Systems
- Capgemini
- LTIMindtree
- Tech Mahindra
- Amdocs
- Cognizant
- Wipro
- Accenture
- Cybage
- BrowserStack
- Druva
- Saksoft
- TCS
- IBM India

Batch Modes and Duration

Duration: 2 months (8 weeks) for the weekday and online tracks; about 10 weeks for the weekend track

Classroom

- Location: Archer Infotech Kothrud campus (Flat No. 12, Divyadarshan Housing Society, Kothrud, Pune 411038)
- Morning batch: Monday–Friday 10:00–11:30
- Evening batch: Monday–Friday 19:00–20:30
- Saturday lab session: 10:00–13:00 (three-hour deep practice)

Live online

- Live sessions: Monday–Friday 19:30–21:00 IST
- Recordings available in the LMS within 24 hours
- Google Meet for live sessions
- GitHub for code distribution
- Batch Slack channel for doubts

Weekend

- Saturday and Sunday 10:00–13:00 (3 hours per day, 6 hours per week)
- The weekend track runs about 10 weeks instead of 8 to keep total contact hours the same

Batch sizes stay under 18 for weekday and online tracks and under 12 at weekends. New batches typically start every three to four weeks; check the live batch schedule.

Fees

The API Testing and Automation track is priced in the mid band of our catalogue, reflecting the two-month duration and the framework build in your chosen stack. EMI plans are available; contact admissions for the current fee structure.

Indicative range: Rs. 22,000 – Rs. 32,000 (typical track band)

- One-time payment (5% discount)
- EMI: 50% at enrolment, 50% at four weeks
- EMI: three-month plan

Placement Support

Placement support is bundled with the course, with no separate fee. Graduates plug into the Archer placement cell built on seventeen years of relationships with Pune hiring managers. The process starts in Week 6, in parallel with your capstone, so your CV and GitHub portfolio are ready by the time you finish.

We do not guarantee placement. Our institute-records placement rate is 90% across tracks. For this track the positioning advice is specific: apply to general QA automation and SDET roles rather than filtering for 'API tester' in the title, and lead with the combination — contract testing, authorisation coverage, CI integration — because that breadth is what moves an offer band.

The placement process

- Week 6: CV review, GitHub portfolio audit and positioning for automation and SDET roles
- Week 7: First mock interview (technical) with structured feedback
- Week 8: Second mock interview (HR and communication) plus an offer-negotiation primer
- Post-completion weeks 1–2: Direct introductions to partner companies hiring automation and integration testers
- Post-completion weeks 3–8: Weekly placement-cell check-ins and interview coaching as you apply

Partner companies

- Persistent Systems
- Capgemini
- LTIMindtree
- Tech Mahindra
- Amdocs
- Cybage
- Cognizant
- Wipro
- Saksoft
- TCS

Prerequisites and How to Start

The course assumes you understand what testing is for and can use a computer confidently. It does not assume programming: Modules 1 to 5 need none at all, and Module 6 teaches the coding required in whichever stack you choose. A typical batch mixes manual testers adding their first automation skill, UI automation testers broadening into the service layer, recent graduates, and a few backend developers taking on test ownership.

Choosing your Module 6 stack early helps, though you can decide once the course starts and you have seen all three. Pick Rest Assured if you are targeting Java teams, pytest if you prefer Python, Playwright if you also want UI automation in the

same framework. None of the starting steps below are gated.

Suggested first steps

- Install Postman and create a free account so collections sync
- Try a public API such as reqres.in or the Swagger Petstore — send one GET and one POST
- Create a free GitHub account and practise the clone, commit and push cycle once
- Open any public Swagger UI page and read how paths, parameters and schemas are described
- Install the runtime for your likely Module 6 stack — Java and Maven, Python 3.12, or Node.js 20

Frequently Asked Questions

Do I need programming experience to join?

Not to start. Modules 1 through 5 — HTTP fundamentals, Postman, authentication, Postman scripting and OpenAPI — need no prior programming. Module 6 then teaches the coding required in whichever automation stack you choose.

Which automation stack should I pick in Module 6?

Rest Assured if you are heading for Java teams, PyTest if you prefer Python or already know it, Playwright if you also want UI automation in the same framework. You build the framework once in your chosen stack; the concepts transfer directly to the other two.

Why learn API testing separately from UI testing?

API tests run in milliseconds, do not break when the interface changes, and can exercise error paths and permission rules that are hard to trigger through a screen. Most defects live at the service layer, and most Pune SDET job descriptions now require both layers.

Does this course cover API security testing?

The fundamentals, framed by the OWASP API Security list — authentication and authorisation vulnerabilities including broken object-level authorisation, input validation, injection basics, rate limiting and sensitive data exposure. It is a tester's working grounding, not a penetration-testing course.

Is Postman enough, or do I need a coding framework?

Postman with Newman in CI is genuinely sufficient for many teams, and you will be able to deliver that by Week 4. A coded framework becomes worth it when you need complex data setup, reuse across suites, or integration with UI tests — which is why the course covers both.

How much of the course is AI-related?

One full module of eighteen topics, plus AI-assisted work in the capstone. API testing is where AI is most immediately useful, because an OpenAPI spec is a machine-readable contract a model can generate cases from — including the boundary cases people skip. The module also covers how to validate that output.

Will I finish with something I can show an interviewer?

Yes. Three projects: a Postman suite runnable via Newman, a layered automation framework in your chosen stack, and a capstone running in CI with contract testing and a documented AI-assisted workflow, all on GitHub.

Should I take this before or after a Selenium or Playwright course?

Either order works. Manual testers often take API testing first because the first half needs no programming. Testers who already automate UI usually take it second, to complete the pairing that SDET roles ask for.

Enquire or book a free demo session. Archer Infotech, Flat No. 12, Divyadarshan Housing Society, behind Kothrud Bus Stand Road, Kothrud, Pune 411038. Phone +91 9850 678451. Email info@archerinfotech.in. Web archerinfotech.in.